

Design Patterns Used in the Application

1. Introduction

This is a cryptocurrency analytics application, built using Flask and microservices architecture. It offers market data, LSTM model and news-based sentiment trading signals.

While developing this, we refactored and reorganized the code so that it adhered to several software design patterns.

Below, it is explained why some of those patterns are good choices for this application, and how they are implemented in the code.

2. Microservices Architecture

The Microservices Architecture pattern is about setting up an application as a bunch of small, separate services. Each service handles just one specific thing and talks to the other services using APIs. It is like breaking down a big job into smaller tasks that different people can manage.

How it is used in this project

Well, in our situation, the system is split up into a few services, including:

- **API Gateway** – handles all frontend requests
- **Market Service** – gives price and on-chain data
- **Prediction Service** – performs LSTM-based price prediction
- **Sentiment Service** – analyzes sentiment and news
- **Database Service** – manages database access

Each service runs independently on a different port.

Why it was used

There were a few good reasons to go with this approach. One is that it keeps each piece of the system simple and easy to focus on. It also makes the code easier to both understand and debug. And another big plus is that you can update one service without messing up or affecting the others.

3. API Gateway Pattern

The API Gateway Pattern provides a single entry point for all client requests; instead of the frontend needing to call a bunch of different services directly, it just communicates to this one gateway.

How it is used in the code

The `api_gateway/app.py` file takes requests and sends them to the right backend services. Here is a snippet:

```
@app.route("/api/coins")

def api_coins():
    return jsonify(requests.get(f"{MARKET_URL}/api/coins").json())
```

So, the frontend never directly accesses the Market or Prediction services.

Why it was used

- Simplifies frontend logic
- Hides internal service details
- Makes future changes safer and easier

4. Facade Pattern

The Facade Pattern provides a simplified interface to a complex system. It is similar to an API Gateway, but with more focus on not letting you see what is going on under the hood. Internal complexity is hidden.

How it is used in the code

Our API Gateway works as this facade. It shows you simple endpoints like:

- `/api/coins`
- `/api/lstm/<symbol>`
- `/api/combined-signal/<symbol>`

Internally, these endpoints may communicate with multiple services.

Example

```
@app.route("/api/combined-signal/<symbol>")
def api_combined_signal(symbol):
    return jsonify(
        requests.get(f"{SENTIMENT_URL}/api/combined-signal/{symbol}").json()
    )
```

Why it was used

- Reduces complexity for the client
- Code is easier to read

- The system becomes more user friendly

5. Layered Architecture Pattern

The Layered Architecture pattern is about dividing up an application into layers, with each layer taking on specific responsibility. It keeps things clean.

How it is used in the Market Service

- **API Layer** – handles HTTP requests
- **Business Logic Layer** – processes data
- **Data Layer** – accesses the database

Example

```
# API layer
@app.route("/api/coins")
def api_coins():
    return jsonify(get_latest_coins())
# Business logic layer
def get_latest_coins():
    rows = db.query(...)
```

Why it was used

- Keeps code organized
- Logic can be used again in other places.
- Easier to modify or extend

6. Singleton Pattern (Database Service)

The Singleton Pattern ensures that only one instance of a class exists during the application's lifetime.

This pattern is ideal for database connections, which should not be created multiple times.

How it is used in the code

The `Database` class is implemented as a Singleton:

```
class Database:
    _instance = None
    _lock = Lock()

    def __new__(cls):
        if cls._instance is None:
            with cls._lock:
```

```

        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.conn = sqlite3.connect(DB_PATH,
check_same_thread=False)
        return cls._instance

```

Why it was used

- Prevents multiple database connections
- Avoids duplicated connection code
- Improves efficiency and consistency

7. Strategy Pattern (Prediction Service)

The Strategy Pattern allows different algorithms to be used interchangeably without changing the main application logic.

How it is used in the code

The prediction logic is encapsulated inside a single function and model class:

```

def run_lstm(symbol):
    model = LSTMModel(input_size)
    ...
    return prediction_result

```

If a different prediction model is needed in the future, it can replace the LSTM without changing the API.

Why it was used

- Makes the prediction logic flexible
- Encourages clean separation
- Improves future extensibility

8. Aggregator Pattern (Combined Signal)

The Aggregator Pattern combines data from multiple sources into one result.

How it is used in the code

The `combined_signal()` function merges:

- Sentiment analysis results
- On-chain market data

```
score = sentiment_score * 0.6 + onchain_strength * 0.4
```

Based on the final score, the system outputs **BUY**, **SELL**, or **HOLD**.

Why it was used

- Centralizes decision logic
- Reduces frontend complexity
- Makes logic easier to tune

9. Model–View–Controller (MVC) Pattern

The MVC pattern separates application logic, data, and presentation.

How it is used in the code

- **Model:** Database and ML models
- **View:** HTML templates
- **Controller:** Flask routes

```
@app.route("/markets")
def markets():
    return render_template("markets.html")
```

Why it was used

- Cleaner frontend code
- Easier UI changes
- Better structure

10. Conclusion

By applying multiple design patterns, this project achieves a clean, modular, and maintainable design.

Each pattern was chosen for a specific reason and directly improves code quality, readability, and future extensibility.

The combination of Singleton, API Gateway, Facade, Strategy, and Aggregator patterns ensures that the application is well-structured and aligned with software engineering best practices.